

INSTITUTO TECNOLÓGICO DE IZTAPALAPA
INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:
TALLER DE BASE DE DATOS

NOMBRE DEL PROFESOR:
FLORES GONZÁLEZ JUAN ALBERTO

PRACTICA

PRESENTA:
GIL CHÁVEZ AXEL OSVALDO
GONZÁLEZ MARTÍNEZ ARATH NATANAEL
GUERRERO BERNAL KEVIN ANDRÉS
OROZCO RODRIGUEZ BRYAN CHISTOPHER
OROZCO RODRIGUEZ SCARLETT GABRIELA

CIUDAD DE MÉXICO 07 / 10

OBJETIVO GENERAL.

Desarrollar e implementar un sistema de control de estacionamiento mediante Flutter, y dart lenguaje de programación para el Instituto Tecnológico de Iztapalapa que permita (guardar, gestionar, y manipular la información mediante un gestor de base de datos) en el gestor MySQL registrar usuarios, vehículos, roles y controlar los accesos de manera segura y ordenada. Con ello se busca optimizar el uso de los espacios disponibles y contar con información confiable sobre quién ingresa con su debido tarjetón.

OBJETIVOS ESPECIFICOS.

1. Diseñar una base de datos relacional que almacene la información de usuarios, roles y vehículos de la comunidad universitaria.
2. Establecer roles de usuario (alumno, docente, administrativo, vigilante y administrador) con permisos diferenciados dentro del sistema.
3. Implementar el registro y la validación de vehículos mediante placa, marca y modelo, vinculados a su propietario, controlando la vigencia y el estatus (activo o inactivo) de cada usuario para regular el acceso al estacionamiento.
4. Diseña en el programa Flutter las interfaces graficas de usuario

PLANTEAMIENTO DEL PROBLEMA.

En el Instituto Tecnológico de Iztapalapa, el control de acceso al estacionamiento se realiza de forma manual y con registros en papel, lo que genera procesos lentos, errores de captura y poca confiabilidad en la información. Esta situación dificulta saber con certeza qué vehículos pertenecen a alumnos, docentes o personal administrativo, y qué personas tienen autorización vigente para ingresar.

Además, la falta de un registro provoca que el estacionamiento se sature, que personas ajenas a la universidad puedan acceder sin ser detectadas y que no exista un historial confiable de entradas y salidas. Esto representa un riesgo para la seguridad de los vehículos y de la comunidad universitaria y complica la labor del personal de vigilancia al momento de verificar identidades y vehículos.

Ante este panorama, surge la necesidad de contar con un sistema digital respaldado por una base de datos estructurada, que concentre los datos de los usuarios, sus roles y sus vehículos, y que permita validar vigencias y estatus de forma rápida.

Tabla Persona

Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción	Valores permitidos / Restricciones
id_persona	INT	5	No	PK	Identificador único del usuario	Autoincremental
nombres	VARCHAR	100	No	—	Nombre(s) de pila del usuario	Solo letras
apellido_p	VARCHAR	100	No	—	Apellido paterno	Solo letras
apellido_m	VARCHAR	100	Sí	—	Apellido materno	Solo letras
teléfono	VARCHAR	15	Sí	—	Número telefónico de contacto	10-15 dígitos
correo	VARCHAR	150	No	UNIQUE	Correo electrónico institucional	Formato válido (usuario@dominio)
matricula	VARCHAR	20	Sí	UNIQUE	Matrícula escolar (solo alumnos)	Nulo si es empleado
no_empleado	VARCHAR	20	Sí	UNIQUE	Número de empleado (solo empleados)	Nulo si es alumno
estatus	ENUM	—	No	—	Estado actual del usuario en el sistema	Activo / Inactivo / Suspendido
genero	ENUM	—	Sí	—	Género del usuario	Masculino / Femenino / Otro
contrasena	VARCHAR	255	No	—	Contraseña cifrada (hash)	Mínimo 8 caracteres, cifrada
fecha	DATE	—	No	—	Fecha de registro en el sistema	Formato AAAA-MM-DD
horario	VARCHAR	50	Sí	—	Horario asignado o permitido para acceso	Ej. "07:00-14:00"
vigencia	DATE	—	No	—	Fecha de vencimiento del acceso	Formato AAAA-MM-DD

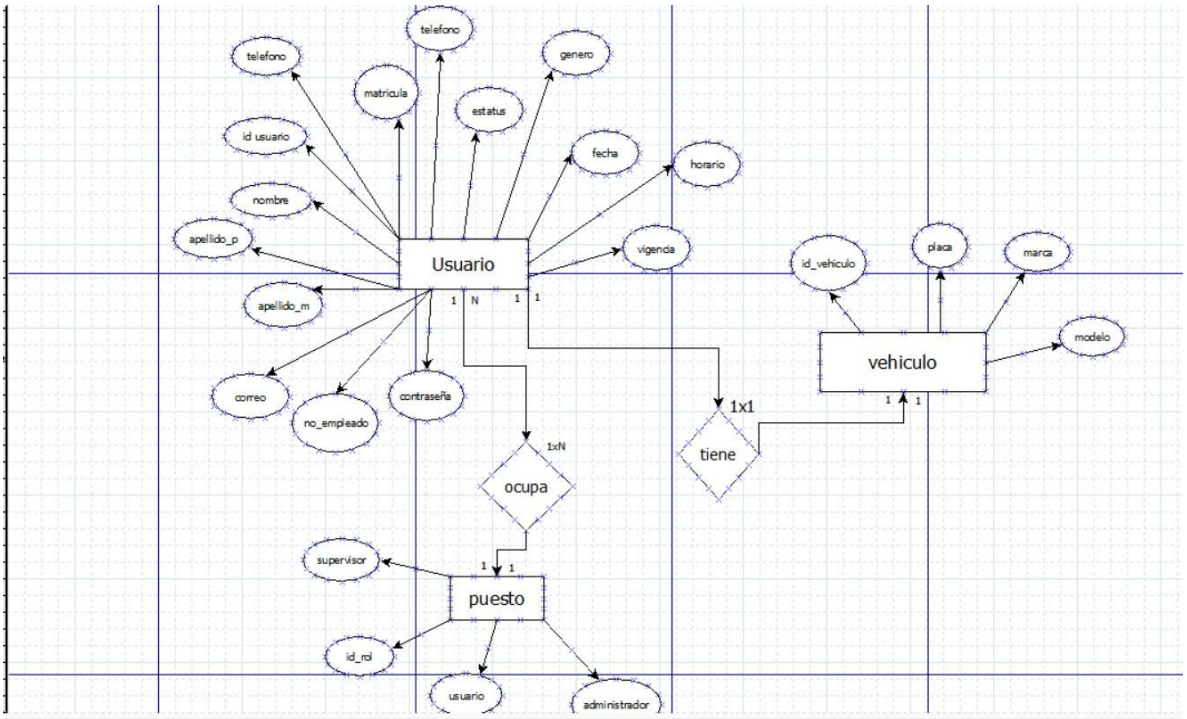
Tabla Vehiculo

Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción	Valores permitidos / Restricciones
id_vehiculo	INT	5	No	PK	Identificador único del vehículo	Autoincremental
placa	VARCHAR	10	No	UNIQUE	Placa de circulación del vehículo	Alfanumérico, sin espacios
marca	VARCHAR	50	No	—	Marca del vehículo	Ej. Nissan, Toyota, Honda
modelo	VARCHAR	50	No	—	Modelo y/o año del vehículo	Ej. Versa 2020
id_usuario	INT	5	No	FK	Usuario propietario del vehículo	Referencia a usuario.id_usuario

Tabla Rol

Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción	Valores permitidos / Restricciones
id_rol	INT	5	No	PK	Identificador único del rol	Autoincremental
usuario	VARCHAR	50	No	—	Rol de usuario común (alumno/empleado)	—
administrador	VARCHAR	50	No	—	Rol con privilegios administrativos	—
supervisor	VARCHAR	50	No	—	Rol de supervisión de accesos	—

Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción
id_usuario	INT	5	No	PK, FK	Referencia a usuario.id_usuario
id_rol	INT	5	No	PK, FK	Referencia a roles.id_rol


telefono
telefono
genero
matricula
estatus
id usuario
fecha
horario
nombre
apellido_p
vigencia
Usuario
apellido_m
1
N
1
1
correo
no_empleado
contraseña
1x1
tiene
1xN
ocupa
supervisor
1
1
puesto
id_rol
usuario
administrador
id_vehiculo
placa
marca
modelo
vehiculo
1
1

ANÁLISIS DE REQUERIMIENTOS DE SISTEMA

Proyecto: Estacionamiento ITIZ

Stack: Flutter / Dart · VS Code · MySQL · PostgreSQL · Android Studio

1. Objetivo

Definir los requisitos de hardware, sistema operativo, dependencias y configuraciones necesarias para instalar, ejecutar y mantener el entorno de desarrollo completo, garantizando compatibilidad, rendimiento y estabilidad.

2. Requerimientos de Sistema Operativo

2.1 Sistemas Operativos Soportados

Plataforma	Versiones Mínimas Recomendadas	Observaciones
Windows	Windows 10 64 bits (versión 1903 o superior) Recomendado: Windows 11 64 bits	Requerido para Android Studio y emuladores; no compatible con versiones de 32 bits
macOS	macOS 10.15 (Catalina) o superior Recomendado: macOS 12+ (Monterey/Ventura/Sonoma)	Necesario si se compila para iOS; requiere procesador Intel o Apple Silicon con Rosetta 2
Linux	Distribuciones con kernel 5.4+ (Ubuntu 20.04 LTS+, Fedora 34+, etc.)	Requiere paquetes adicionales de 32 bits para emuladores Android

Restricción clave: Todo el stack requiere arquitectura de 64 bits exclusivamente. Sistemas de 32 bits no son compatibles con Flutter, Android Studio ni las versiones actuales de los gestores de bases de datos.

2.2 Requerimientos Específicos por Componente

Componente	Requerimiento de SO	Detalle
Flutter / Dart	Windows 10+ / macOS 10.14+ / Ubuntu 18.04+	Flutter 3.x requiere estos mínimos; versiones anteriores ya no reciben soporte
Android Studio	Windows 10+ / macOS 10.14+ / Ubuntu 18.04+	El emulador requiere aceleración por hardware habilitada en el SO
VS Code	Windows 10+ / macOS 10.11+ / Ubuntu 16.04+	Soporte amplio, pero se recomienda SO actualizado para extensiones de Flutter
MySQL 8.0+	Windows 10+ / macOS 10.13+ / Ubuntu 18.04+	Servidor y cliente; en Windows requiere Visual C++ Redistributable
PostgreSQL 14+	Windows 10+ / macOS 10.13+ / Ubuntu 20.04+	Versiones LTS para estabilidad en desarrollo

3. Requerimientos de Hardware

Recurso	Mínimo	Recomendado	Justificación
Procesador	Dual-core a 2.0 GHz (Intel Core i3 / AMD equivalente)	Quad-core o superior (Intel Core i5/i7 / AMD Ryzen 5/7)	Emulador Android + compilación Flutter consumen múltiples núcleos
Memoria RAM	8 GB	16 GB o más	Android Studio ~3–4 GB; Emulador ~2–3 GB; VS Code + DB + navegador ~3–4 GB; 8 GB sufre saturación
Almacenamiento	10 GB libres	30–50 GB en SSD	Android Studio + SDK ~8 GB; Flutter + dependencias ~3 GB; Bases de datos, emuladores, archivos de proyecto crecen rápidamente
Pantalla	1280 × 800	1920 × 1080 o superior	Interfaz de Android Studio y herramientas de desarrollo requieren espacio
Puertos/Conectividad	—	USB 3.0, conexión a internet estable	Conexión con dispositivos físicos móviles; descarga de paquetes y dependencias

4.3 Configuración de Red y Puertos

Aplicación	Puertos por defecto	Propósito
MySQL	3306	Conexión al servidor de BD
PostgreSQL	5432	Conexión al servidor de BD
Emulador Android	Rango dinámico	Depuración de app móvil
Servicios de desarrollo	8080, 5000, 8000 (ejemplo)	Backend/APIs

Deben estar libres o permitidos en el firewall local.

4.4 Extensiones de VS Code Obligatorias

- Flutter → soporte de lenguaje, compilación, depuración
- Dart → resaltado de sintaxis, análisis estático
- Flutter Snippets → agilizar desarrollo
- MySQL / PostgreSQL (extensiones de cliente) → administración de BD desde el editor

5. Requerimientos Funcionales del Entorno

1. Compilación multiplataforma: Flutter debe generar aplicaciones nativas para Android (y opcionalmente iOS/macOS/Windows/Linux) desde el mismo código base.
2. Ejecución y depuración: Soporte para emulador Android y dispositivos físicos conectados por USB/Wi-Fi, con puntos de interrupción y consola en tiempo real.
3. Persistencia de datos: Instalación simultánea y operativa de MySQL y PostgreSQL sin conflictos de servicio ni puertos.
4. Integración completa: VS Code como IDE principal con capacidad de invocar herramientas de Flutter, ejecutar comandos de BD y lanzar Android Studio cuando sea necesario.
5. Actualizaciones: El sistema debe permitir actualizar Flutter, Dart, SDK de Android y gestores de BD sin romper la compatibilidad.

6. Requerimientos No Funcionales

Aspecto	Requerimiento
Estabilidad	El entorno debe permanecer estable durante compilaciones largas y sesiones de desarrollo continuas
Rendimiento	La compilación de cambio debe ser menor a 2 segundos en equipos con configuración recomendada
Seguridad	Acceso a bases de datos mediante usuario y contraseña; no exponer servicios a redes públicas sin configuración específica
Portabilidad	El entorno debe poder replicarse en equipos distintos con la misma configuración
Mantenibilidad	Versiones identificadas y compatibles: Flutter 3.22+, Dart 3.4+, Android Studio Iguana/Jellyfish+, MySQL 8.0+, PostgreSQL 15+

7. Recomendaciones Finales

- RAM ≥ 16 GB es indispensable para trabajar fluido con todos los componentes a la vez.
- Preferir SSD sobre disco mecánico: reduce tiempos de compilación en un 40–60 %.
- En Windows, usar WSL 2 si se requiere entorno Linux junto con el nativo.
- Mantener una copía de seguridad de la configuración de VS Code y las bases de datos de desarrollo.

Avance de GanttProject

